

Практика по алгоритмам

Евгений Кравченко, Виктор Крыштапович, Александр Мишунин *

Осень, 2025

Содержание

1	Асимптотика	3
1.1	Практика	3
1.2	Домашнее задание	6
2	Линейные алгоритмы	7
2.1	Практика	7
2.2	Домашнее задание	9
3	Разделяй и властвуй	10
3.1	Практика	10
3.2	Домашнее задание	12
4	Сортировки и кучи	13
4.1	Практика	13
4.2	Домашнее задание	15
5	Шустрая сортировка	17
5.1	Практика	17
5.2	Домашнее задание	18
6	Сортировки и порядковые статистики	19
6.1	Практика	19
6.2	Домашнее задание	20
7	Демоническое программирование	21
7.1	Практика	21
7.2	Домашнее задание	22
8	Динамическое программирование #2	23
8.1	Практика	23
8.2	Домашнее задание	24
9	Жадные алгоритмы	25
9.1	Практика	25
9.2	Домашнее задание	26
10	DSU и остовные деревья	28
10.1	Практика	28
10.2	Домашнее задание	30

*Составители сборника не всегда являются авторами задач. Авторы не указаны в учебных целях.

11 DFS	31
11.1 Практика	31
11.2 Домашнее задание	32
12 BFS	33
12.1 Практика	33
12.2 Домашнее задание	34
13 В предыдущих сериях	35
13.1 Практика	35
13.2 Домашнее задание	37

1 Асимптотика

1.1 Практика

Пусть f, g — функции $\mathbb{N} \rightarrow \mathbb{N}$ или $\mathbb{N} \rightarrow \mathbb{R}_{>0}$ (далее будет ясно из контекста, какой класс функций используется). Напомним определения:

- $f \in \mathcal{O}(g) \equiv \exists N, C > 0 : \forall n \geq N : f(n) \leq C \cdot g(n)$
- $f \in \Omega(g) \equiv \exists N, C > 0 : \forall n \geq N : C \cdot g(n) \leq f(n)$
- $f \in \Theta(g) \equiv \exists N, C_1 > 0, C_2 > 0 : \forall n \geq N : C_1 \cdot g(n) \leq f(n) \leq C_2 \cdot g(n)$
- $f \in o(g) \equiv \forall C > 0 : \exists N : \forall n \geq N : f(n) < C \cdot g(n)$
- $f \in \omega(g) \equiv \forall C > 0 : \exists N : \forall n \geq N : C \cdot g(n) < f(n)$

В дальнейшем, когда речь идет о принадлежности функций вышеопределенным множествам, мы будем использовать знак “=” вместо “ $\in$ ”, т.к. в литературе обычно используются именно такие обозначения.

1. Докажите, что:

- (a) $f(n) = \Omega(g(n)) \Leftrightarrow g(n) = \mathcal{O}(f(n))$
- (b) $f(n) = \omega(g(n)) \Leftrightarrow g(n) = o(f(n))$
- (c) $f(n) = \Theta(g(n)) \Leftrightarrow f(n) = \mathcal{O}(g(n)) \wedge f(n) = \Omega(g(n))$

2. **Контекст имеет значение**

Правда ли, что $f(n) = \mathcal{O}(f(n)^2)$?

3. **Несколько аргументов**

Придумайте определение для $f(n, m) = \mathcal{O}(g(n, m))$.

4. **Асимметрия**

- (a) Правда ли, что $\min(f(n), g(n)) = \Theta(f(n) + g(n))$?
- (b) Правда ли, что $\max(f(n), g(n)) = \Theta(f(n) + g(n))$?

5. **Классы**

Определим отношение “ $\sim$ ”. Будем говорить, что $f \sim g$, если $f = \Theta(g)$. Покажите, что $\sim$ — отношение эквивалентности, т.е. оно

- Рефлексивное: $\forall f : f \sim f$,
- Симметричное: $\forall f, g : f \sim g \Leftrightarrow g \sim f$,
- Транзитивное: $\forall f, g, h : (f \sim g) \wedge (g \sim h) \Rightarrow f \sim h$.

6. **Порядки**

Определим отношение “ $\preceq$ ”. Будем говорить, что $f \preceq g$, если $f = \mathcal{O}(g)$.

- (a) Правда ли, что $\preceq$ — отношение предпорядка (рефлексивное и транзитивное)?
- (b) Правда ли, что $\preceq$ — отношение частичного порядка (+ антисимметричность)?
- (c) Правда ли, что $\preceq$ — отношение частичного порядка на классах эквивалентности по $\sim$?

7. Правда ли, что если $y(n)$ — монотонная неограниченная функция, и $f(n) = \mathcal{O}(g(n))$, то $f(y(n)) = \mathcal{O}(g(y(n)))$?

- 8. (a) Требуется реализовать очередь с амортизированным временем работы всех операций $\mathcal{O}(1)$, используя $\mathcal{O}(1)$ стеков.
- (b) Придумайте стек, в котором можно узнавать минимум за $\mathcal{O}(1)$. Все остальные операции стека также должны работать за $\mathcal{O}(1)$.

- (c) Придумайте очередь, в которой можно узнавать минимум за амортизированное $\mathcal{O}(1)$.
Все остальные операции очереди также должны работать за амортизированное $\mathcal{O}(1)$.

9. Считайте здесь, что функции здесь $\mathbb{N} \rightarrow \mathbb{N}$ и что $\forall n : f(n) > 1 \wedge g(n) > 1$.

- (a) $f(n) = \Omega(f(n/2))$?
 (b) $f(n) = \mathcal{O}(g(n)) \Rightarrow \log f(n) = \mathcal{O}(\log g(n))$?
 (c) $f(n) = \mathcal{O}(g(n)) \Rightarrow 2^{f(n)} = \mathcal{O}(2^{g(n)})$?
 (d) $f(n) = o(g(n)) \Rightarrow \log f(n) = o(\log g(n))$?
 (e) $f(n) = o(g(n)) \Rightarrow 2^{f(n)} = o(2^{g(n)})$?
 (f) $\sum_{k=1}^n \frac{1}{k} = \Omega(\log n)$?

10. Оцените время работы следующих программ:

(a)

```
for (a = 1; a < n; a++)
  for (b = 0; b < n; b += 1)
    {}
```

(b)

```
for (a = 1; a < n; a++)
  for (b = 0; b < n; b += a)
    {}
```

- (c) Найти такие $a, b, c \in \mathbb{N} : abc = n, a + b + c = \min$. Решение:

```
for (a = 1; a <= n; ++a)
  for (b = 1; a * b <= n; ++b)
    c = n / a / b, ... ;
```

- (d) Еще одно решение (c):

```
for (a = 1; a * a * a <= n; ++a)
  for (b = 1; b * b <= n; ++b)
    c = n / a / b, ... ;
```

- (e) И еще одно решение (c):

```
for (a = 1; a * a * a <= n; ++a)
  for (b = a; a * b * b <= n; ++b)
    c = n / a / b, ... ;
```

- (f) Дополнительный вопрос: что делает этот код?

```
a = 1, b = n;
while (a < b) {
  while (x[a] < M && a <= b) a++;
  while (x[b] > M && a <= b) b--;
  if (a <= b) swap(x[a++], x[b--]);
}
```

- (g) Дополнительный вопрос: а если бы вместо 2 было бы 1?

```
while (a >= 2)
  a = sqrt(a);
```

- (h) Решето Эратосфена (пользуемся, что: $p_n \approx n \ln n$)

```

for (p = 2; p < n; p++)
  if (min_divisor[p] == 0) // is prime
    for (x = p + p; x < n; x += p)
      if (min_divisor[x] == 0)
        min_divisor[x] = p;

```

Дополнительные задачи

11. Дан массив целых чисел от 1 до n длины $n + 1$, который нельзя модифицировать. Используя $\mathcal{O}(\log n)$ битов дополнительной памяти, найдите в массиве пару одинаковых чисел за $\mathcal{O}(n)$.
12. Дана последовательность $\sigma = \langle a_1, a_2, \dots, a_m \rangle$, где каждый $a_i \in [n] = \{1, 2, \dots, n\}$. Обозначим частоту появления элемента x через $f_\sigma[x] = |\{i \mid a_i = x\}|$. Известно, что $\exists x : f_\sigma[x] = 1$ и для всех остальных значений $y \neq x, f_\sigma[y] \equiv 0 \pmod 2$. Требуется найти x за один проход по последовательности, используя $\mathcal{O}(\log n + \log m)$ бит памяти.
13. Дана последовательность $\sigma = \langle a_1, a_2, \dots, a_m \rangle$, где каждый $a_i \in [n]$. Требуется проверить, правда ли, что $\exists x : f_\sigma[x] > \frac{m}{2}$, и если такой x есть, то найти его за один проход по последовательности. Докажите, что любое решение потребует $\Omega(m \cdot (\log n - \log m + 1))$ бит памяти.
14. Разрешим сделать два прохода по последовательности. Решите прошлую задачу за $\mathcal{O}(\log n + \log m)$ бит памяти.

1.2 Домашнее задание

1. Эквивалентны ли следующие факты?

- $f = \Theta(g)$
- $\exists C, 0 < C < \infty : \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = C$

Докажите, или приведите контрпример (в обе стороны).

2. Дайте ответ для двух случаев $\mathbb{N} \rightarrow \mathbb{N}$ и $\mathbb{N} \rightarrow \mathbb{R}_{>0}$:

- Если в определении $\mathcal{O}$ опустить условие про N (т.е. оставить просто $\forall n$), будет ли полученное определение эквивалентно исходному?
- Тот же вопрос про o .

3. Продолжим отношение " $\preceq$ " на функциях до отношения на классах эквивалентности по отношению эквивалентности " $\sim$ ", введённому на практике. Правда ли, что получится отношение *линейного порядка* (то есть $\forall f, g : (f \preceq g) \vee (g \preceq f)$)?

4. Докажите, или приведите контрпример:

- $g(n) = o(f(n)) \Rightarrow f(n) + g(n) = \Theta(f(n))$
- $f(n) = \mathcal{O}(g(n)) \Leftrightarrow f(n) = o(g(n)) \vee f(n) = \Theta(g(n))$

5. Заполните табличку и поясните (особенно строчки 4 и 7):

A	B	$\mathcal{O}$	o	Θ	ω	Ω
n	n^2	+	+	—	—	—
$\log^k n$	n^ϵ					
n^k	c^n					
$\sqrt{n}$	$n^{\sin n}$					
2^n	$2^{n/2}$					
$n^{\log m}$	$m^{\log n}$					
$\log(n!)$	$\log(n^n)$					

Здесь все буквы, кроме n , — положительные константы.

Дополнительные задачи

6. Считайте здесь, что функции здесь $\mathbb{N} \rightarrow \mathbb{N}$ и что $\forall n : f(n) > 1 \wedge g(n) > 1$. Правда ли, что:

- $f(n) = \mathcal{O}(g(n)) \Rightarrow \log f(n) = \mathcal{O}(\log g(n))$?
- $f(n) = o(g(n)) \Rightarrow \log f(n) = o(\log g(n))$?
- $f(n) = o(g(n)) \Rightarrow 2^{f(n)} = o(2^{g(n)})$?

7. Упорядочите функции по скорости роста и обозначьте неравенства между соседями. Укажите, в каких неравенствах $f = o(g)$, а в каких $f = \Theta(g)$

$\log(\log^* n)$	$2^{\log^* n}$	$(\sqrt{n})^{\log n}$	n^2	$n!$	$(\log n)!$
$(3/2)^n$	n^3	$\log^2 n$	$\log n!$	2^{2^n}	$n^{1/\log n}$
$\ln \ln n$	$\log^* n$	$n \cdot 2^n$	$n^{\log \log n}$	$\ln n$	1
$2^{\ln n}$	$(\log n)^{\log n}$	e^n	$4^{\log n}$	$(n+1)!$	$\sqrt{\log n}$
$\log^* \log n$	$2^{\sqrt{2 \log n}}$	n	2^n	$n \log n$	$2^{2^{n+1}}$

Примечание: $\log^*(n) = \begin{cases} 0 & \text{если } n \leq 1; \\ 1 + \log^*(\log n) & \text{иначе.} \end{cases}$

Докажите попарные сравнения в вашем порядке (и пронумеруйте, чтобы на них удобно было ссылаться).

2 Линейные алгоритмы

2.1 Практика

- (а) Покажите методом потенциалов, что очередь на двух стеках (как в предыдущей практике) начального размера n выполняет m операций за $\mathcal{O}(n + m)$ времени.
(б) Придумайте более эффективный на практике вариант очереди с минимумом на основе пары из обычной очереди и дека.

- Дана последовательность, составленная из скобок '(', ')', '[', ']', '{', '}'. Последовательность называется правильной, если она порождается грамматикой:

$$\begin{aligned} S &\rightarrow \varepsilon \\ S &\rightarrow (S)S \\ S &\rightarrow [S]S \\ S &\rightarrow \{S\}S \end{aligned}$$

Например, последовательности '([{}])' и '()()' правильные, а '[()]' и '[({})]' – нет.

Придумайте алгоритм, который проверяет правильность последовательности за $\mathcal{O}(n)$.

- Дан массив целых чисел a_i . Придумайте структуру данных, которая бы умела отвечать на online запросы вида “По данным l и r вернуть $\sum_{i=l}^r a_i$ ” за $\mathcal{O}(1)$. Значения в массиве не меняются. Разрешается сделать предподсчёт за $\mathcal{O}(n)$.
- Дана последовательность $a_1, a_2, \dots, a_n \in \mathbb{Z}$. Найти l, r ($1 \leq l \leq r \leq n$) такие, что сумма $\sum_{i=l}^r a_i$ была бы максимальной.
 - $\mathcal{O}(n)$.
 - $\mathcal{O}(n)$ за 1 проход с $\mathcal{O}(1)$ дополнительной памяти (2 способа).
- Дана последовательность $a_1, a_2, \dots, a_n \in \mathbb{N}$ и $S \in \mathbb{N}$. Найти l, r ($1 \leq l \leq r \leq n$) такие, что сумма $\sum_{i=l}^r a_i = S$. $\mathcal{O}(n)$.
- Вам дан массив из n целых чисел из $[1 \dots n]$ и число k . Найдите такие l и r , что на отрезке $[l, r]$ встречается хотя бы k различных элементов, или сообщите, что такого отрезка нет. Если таких отрезков несколько, выберите тот из них, длина которого минимальна. Время работы $\mathcal{O}(n)$.
- Дана последовательность $a_1, a_2, \dots, a_n \in \mathbb{N}$. Для каждого a_i найти самый правый из элементов, которые левее и не больше его. $\mathcal{O}(n)$.
- Дана последовательность $a_1, a_2, \dots, a_n \in \mathbb{N}$. Найти l, r ($1 \leq l \leq r \leq n$) такие, что
 - значение $(r - l + 1) \min_{i \in [l, r]} a_i$ было бы максимально.
 - значение $\left(\sum_{i \in [l, r]} a_i \right) \min_{i \in [l, r]} a_i$ было бы максимально.

Задачу требуется решить за линейное от n время.

Дополнительные задачи

- Дана последовательность $\sigma = \langle a_1, a_2, \dots, a_m \rangle$, где каждый $a_i \in [n] = \{1, 2, \dots, n\}$. Обозначим частоту появления элемента x через $f_\sigma[x] = |\{i | a_i = x\}|$. Известно, что $\exists x : f_\sigma[x] > \frac{m}{2}$. Требуется найти x за один проход по последовательности, используя $\mathcal{O}(1)$ дополнительной памяти.
- Придумайте очередь на основе $\mathcal{O}(1)$ стеков, у которой все операции работают за **детерминированные** $\mathcal{O}(1)$.

11. Дана квадратная матрица из нулей и единиц. Найти наибольший по площади подпрямоугольник, состоящий только из нулей за $\mathcal{O}(n^2)$.
12. Дана строка s из скобок, а также число k . Для всех i проверить, что подстрока $s[i..i + k]$ является правильной скобочной последовательностью. $\mathcal{O}(n)$. Скобки только одного типа.
13. Вам поступает последовательность online запросов вычислить неизвестную функцию $F : X \rightarrow \{0, 1\}$, после каждого запроса вам говорят правильно ли вы ответили. У вас есть набор детерминированных функций-оракулов $f_1 \dots f_n : X \rightarrow \{0, 1\}$, решающих задачу с ошибками, про которые вы больше ничего не знаете. Придумайте алгоритм, который делает $\mathcal{O}(\log n + m)$ ошибок, где m — число ошибок, которое делает лучший оракул (подсказка: назначьте оракулам веса).
14. Вам дан массив натуральных чисел и число k . Требуется найти подотрезок массива такой, что НОК чисел на нем равен k или заявить, что такого нет. Время работы: $\mathcal{O}(nT_{LCM}(k))$, где $T_{LCM}(k)$ — время подсчета НОК для чисел размера k .

2.2 Домашнее задание

1. (кроме группы Крыштаповича)

Дано двоичное число, заданное в виде битового массива a длины n (от младших битов к старшим), к которому затем m раз прибавляют единицу простейшим алгоритмом:

```
int carry = 1;
for (int i = 0; carry > 0 && i < n; i++) {
    carry += a[i];
    a[i] = carry % 2;
    carry = carry / 2;
}
```

Числа выбраны так, что переполнения не происходит. Докажите **методом потенциалов**, что суммарное время работы будет $\mathcal{O}(n + m)$.

2. Вам дан массив из n элементов и список из m запросов $add(x, l, r)$: прибавить x к каждому элементу на отрезке $[l, r]$. За $\mathcal{O}(n + m)$ выведите массив, получающийся из исходного после выполнения заданных запросов.
3. (кроме группы Кравченко)
Дано число, представленное n цифрами в d -ичной записи без ведущих нулей. Из числа требуется вычеркнуть ровно k цифр так, чтобы результат был максимальным. Придумайте алгоритм и докажите, что он правильно работает. $\mathcal{O}(n)$, d и k — не константы.
4. (кроме группы Крыштаповича)
Вам дана строка из трёх типов скобок. Найдите самую длинную ее подстроку, являющуюся правильной скобочной последовательностью (и докажите, что ваш алгоритм корректно работает). $\mathcal{O}(n)$.
5. (кроме группы Мишунина)
Дан массив целых чисел длины n . Найдите пару непересекающихся отрезков массива с максимальной суммой. $\mathcal{O}(n)$ времени, $\mathcal{O}(1)$ памяти.
6. Дан массив целых чисел длины n . Найдите подотрезок длины от L до R с максимальной суммой. $\mathcal{O}(n)$. Подсказка: используйте префиксные суммы и очередь с минимумом.
7. (только группа Мишунина)
Дан массив длины n с целыми числами a_i , $-n \leq a_i \leq n$. Найдите подотрезок с максимальной суммой, длины от L до R , содержащий от A до B различных чисел, за $\mathcal{O}(n)$. Подсказка: используйте идею из предыдущей задачи и два указателя.

Дополнительные задачи

8. Дана квадратная матрица из нулей и единиц. Найти наибольший по площади подпрямоугольник, состоящий только из нулей за $\mathcal{O}(n^2)$.
9. Вам поступает последовательность online запросов вычислить неизвестную функцию $F : X \rightarrow \{0, 1\}$, после каждого запроса вам говорят правильно ли вы ответили. У вас есть набор детерминированных функций-оракулов $f_0, f_1, \dots, f_{n-1} : X \rightarrow \{0, 1\}$, решающих задачу с ошибками, про которые вы больше ничего не знаете. Оракул f_i делает на всем наборе входных запросов e_i ошибок (вы не знаете e_i). Придумайте алгоритм, который делает $\mathcal{O}(\log n + \min_i e_i)$ ошибок. Подсказка: назначьте оракулам веса.

3 Разделяй и властвуй

3.1 Практика

Пусть $a \geq 1$ и $b > 1$ — константы, $f(n)$ — функция, $T(n)$ определено при неотрицательных n формулой

$$T(n) = aT(n/b) + f(n),$$

где под n/b понимается либо $\lceil n/b \rceil$, либо $\lfloor n/b \rfloor$. Тогда

- если $f(n) = \mathcal{O}(n^{\log_b a - \epsilon})$ для некоторого $\epsilon > 0$, то $T(n) = \Theta(n^{\log_b a})$;
 - если $f(n) = \Theta(n^{\log_b a})$, то $T(n) = \Theta(n^{\log_b a} \log n)$;
 - если $f(n) = \Omega(n^{\log_b a + \epsilon})$ для некоторого $\epsilon > 0$ и если $af(n/b) \leq cf(n)$ для некоторой константы $c < 1$ и достаточно больших n , то $T(n) = \Theta(f(n))$.
1. Дана монотонно неубывающая функция $[1 \dots n] \rightarrow \{0, 1\}$. Требуется алгоритм, который по функции находит последний 0 и первую 1 (если они есть).
 - (a) Придумайте алгоритм, делающий $\mathcal{O}(\log n)$ вызовов функции, и напишите псевдокод.
 - (b) Покажите, что не существует детерминированного алгоритма, делающего менее $\lceil \log(n+1) \rceil$ вызовов функции на всех входах.
 2. Есть n веревок, каждая имеет целую длину l_i , которые можно резать. Нужно получить k одинаковых кусков максимальной целочисленной длины (также могут остаться неиспользованные обрезки). $\mathcal{O}(n \log l_{\max})$.
 3. Сделайте предподсчет за $\mathcal{O}(n \log n)$, чтобы за $\mathcal{O}(\log n)$ online отвечать на запрос “сколько раз число x встречается на отрезке $[l..r]$ ”?
 4. Есть k отсортированных массивов. В сумме массивы содержат n элементов. Слить массивы за $\mathcal{O}(n \log k)$.
 5. Даны два отсортированных массива длины n , которые нельзя модифицировать. Найдите k -ю порядковую статистику в объединении массивов, используя $\mathcal{O}(1)$ дополнительной памяти.
 - (a) За $\mathcal{O}(\log n \cdot \log \text{MAX})$. Значения элементов по модулю не превышают MAX .
 - (b) За $\mathcal{O}(\log n)$.
 6. Даны два отсортированных массива a и b длины n , выведите все попарные суммы $a_i + b_j$ в отсортированном порядке.
 - (a) За $\mathcal{O}(n^2 \log n)$.
 - (b) За $\mathcal{O}(n^3)$ с использованием $\mathcal{O}(n)$ дополнительной памяти.
 - (c) За $\mathcal{O}(n^3)$ с использованием $\mathcal{O}(1)$ дополнительной памяти.

Дополнительные задачи

7. Инверсией в массиве чисел $a[\dots]$ называется такая пара индексов i, j , что $i < j$, но $a_i > a_j$. Дан массив из n различных элементов. Требуется найти число инверсий за $\mathcal{O}(n \log n)$.
8. Найти отрезок с максимальным средним арифметическим длины от L до R . $\mathcal{O}(n \log \text{MAX})$.
9. Определить асимптотику:
 - (a) $T(n) = T(\lceil \frac{n}{2} \rceil) + 1$
 - (b) $T(n) = 4 \cdot T(\lfloor \frac{n}{2} \rfloor) + n^k$ для $k \in \{1, 2, 3\}$.
 - (c) $T(n) = T(a) + T(n-a) + n$ для произвольной константы a .

- (d) $T(n) = 2 \cdot T(\lfloor \frac{n}{2} \rfloor + 17) + n$.
 - (e) $T(n) = T(\alpha \cdot n) + T((1 - \alpha) \cdot n) + n$ для произвольной константы $\alpha \in (0, 1)$.
 - (f) $T(n) = 2 \cdot T(\sqrt{n}) + \log n$
10. Дано множество из n точек на плоскости. Найти пару ближайших точек за $\mathcal{O}(n \log n)$.
11. Найти количество AVL деревьев высоты h из n вершин по простому модулю p за $\mathcal{O}(hn^{\log_2 3})$ (AVL дерево — корневое двоичное дерево, в котором у каждой вершины высоты двух дочерних поддеревьев отличаются не более, чем на 1).

3.2 Домашнее задание

1. Пусть:

$$T(n) = \begin{cases} 1, & n \leq 1 \\ 2T(\lfloor \log n \rfloor) + 2^{\log^* n}, & n > 1. \end{cases}$$

Покажите, что $T(n) = \Theta(2^{\log^* n} \log^* n)$.

Здесь $\log^* n$ — **итерированный логарифм**. Логарифмы в определении $\log^* n$ и $T(n)$ двоичные. Подсказка: не забудьте оценить $T(n)$ снизу. Также, может оказаться полезным доказать лемму:

$$\lfloor \log \lfloor x \rfloor \rfloor = \lfloor \log x \rfloor$$

для $x \geq 1$.

2. Есть m стойл с координатами $x_1, \dots, x_m$ и n коров. Расставить коров по стойлам (не более одной в стойло) так, чтобы минимальное расстояние между коровами было максимально. $\mathcal{O}(m(\log m + \log x_{\max}))$.
3. Пусть алгоритм принимает на вход массивы **различных** целых чисел. Докажите, что следующие условия никогда не выполняются вместе:
- Алгоритм находит максимум (он там один) массива на **любом** входе используя только сравнения на $\leq$.
 - Существует вход длины n на котором алгоритм делает менее $n - 1$ сравнений.
4. Найти второй максимум в массиве за $n + \mathcal{O}(\log n)$ сравнений.
5. Структура данных «файл последовательного доступа» поддерживает следующие операции за $\mathcal{O}(1)$:
- *Read()*: чтение числа из файла на текущей позиции и перевод позиции вперёд на 1 элемент (если конец файла не достигнут).
 - *Write(x)*: запись числа в файл в текущую позицию и перевод позиции вперёд на 1 элемент.
 - *Rewind()*: перевод позиции на начало файла.

Требуется отсортировать файл за $\mathcal{O}(n \log n)$ используя $\mathcal{O}(1)$ дополнительных ячеек памяти и $\mathcal{O}(1)$ дополнительных файлов.

Дополнительные задачи

6. Дан массив a из n различных элементов. Для каждого i найдите $\sum_{j < i} \mathbb{I}[a_j > a_i]$ и $\sum_{j > i} \mathbb{I}[a_i > a_j]$. $\mathcal{O}(n \log n)$.
7. Дано $2n - 1$ коробок с чёрными и белыми шарами. В i -ой коробке находится w_i белых и b_i чёрных шаров. Всего в коробках находится W белых и B чёрных шаров. Требуется выбрать n коробок таким образом, чтобы суммарное число белых шаров в них было не менее $\frac{W}{2}$, а чёрных не менее $\frac{B}{2}$. Решить за $\mathcal{O}(n \log n)$.
8. Дан массив из $2n$ различных чисел. Найдите минимальное и максимальное за $3n - 2$ сравнения и докажите, что это точная нижняя оценка, то есть меньшего количества сравнений может не хватить.
9. Найдите второй максимум в массиве за $n + \lceil \log_2 n \rceil - 2$ сравнение и докажите, что это точная нижняя оценка, то есть меньшего количества сравнений может не хватить.

4 Сортировки и кучи

4.1 Практика

1. Покажите, что любая сортировка, которая верно работает хотя бы на доле $\frac{1}{100^n}$ от всех перестановок, не может работать за $o(n \log n)$ на всех входах.
2. Модифицируйте операцию **SiftUp** для бинарной кучи так, чтобы она по-прежнему работала за $\mathcal{O}(\log n)$, но при этом делала лишь $\mathcal{O}(\log \log n)$ сравнений.
3. Модифицируйте операцию **SiftDown** для бинарной кучи так, чтобы она по-прежнему работала за $\mathcal{O}(\log n)$, но при этом делала лишь $\log_2 n + \mathcal{O}(\log \log n)$ сравнений.
4. Придумайте детерминированную структуру данных на основе бинарной кучи, которая умеет делать **Insert(x)**, **DeleteMedian()** за $\mathcal{O}(\log n)$, **GetMedian()** за $\mathcal{O}(1)$.
5. Дана обычная бинарная куча (с минимумом в голове), требуется узнать k -й минимум.
 - (a) $\mathcal{O}(k \log n)$
 - (b) $\mathcal{O}(k^2)$
 - (c) $\mathcal{O}(k \log k)$
6. Дан массив длины n , в котором встречаются $m \leq n$ различных элементов.
 - (a) Пусть зафиксирован набор частот элементов $p_i > 0, i = 1 \dots m$. Докажите нижнюю оценку $n \left(\sum_{i=1}^m p_i \log \frac{1}{p_i} \right) - n \log e$ на число сравнений в худшем случае при сортировке сравнениями. Полезный факт: $n \ln n \geq \ln n! = n \ln n - n + \mathcal{O}(\ln n)$.
 - (b) Докажите нижнюю оценку $n \log \frac{m}{e}$ на число сравнений в худшем случае при сортировке сравнениями в пределе $n \gg m$.

Дополнительные задачи

7. Пусть B_n (биномиальное дерево порядка n) определено следующим образом:
 - при $n = 0$ это дерево из одной вершины.
 - при $n > 0$ это B_{n-1} , которому первым ребенком подвешено еще одно B_{n-1} .
 - (a) Докажите, что B_n имеет высоту n .
 - (b) Докажите, что в B_n содержится ровно 2^n вершин.
 - (c) Определим биномиальную кучу как набор биномиальных деревьев, в котором нет двух деревьев одного порядка. Покажите, что для любого n существует биномиальная куча с n вершинами.
 - (d) Пусть на всех деревьях биномиальной кучи выполняется свойство кучи (min в голове). Придумайте **GetMin** за $\mathcal{O}(\log n)$.
 - (e) Придумайте **Merge** за $\mathcal{O}(\log n)$.
 - (f) Придумайте **Add** за $\mathcal{O}(\log n)$.
 - (g) Придумайте **ExtractMin** за $\mathcal{O}(\log n)$.
 - (h) Придумайте **DecreaseKey** по ссылке на узел. $\mathcal{O}(\log n)$.
 - (i) Придумайте **Delete** по ссылке на узел. $\mathcal{O}(\log n)$.
8. Дано бинарное дерево: $Tree ::= Node(Tree, Tree) | Empty$ (эта запись означает, что дерево — это либо вершина с парой потомков-деревьев, либо особое значение *Empty*). Определим функцию **rank(x)** следующим образом:
 - **rank(Empty)** = 0

- $\text{rank}(\text{Node}(\text{left}, \text{right})) = \min(\text{rank}(\text{left}), \text{rank}(\text{right})) + 1.$

Назовём бинарное дерево *скошенным влево (левацким)*, если для его вершин выполнено следующее свойство:

$$\forall x = \text{Node}(\text{left}, \text{right}) \text{rank}(\text{left}) \geq \text{rank}(\text{right}).$$

Скошенная влево (левацкая) куча — это скошенное влево дерево, в вершинах которого хранятся данные, для которых выполнено свойство кучи.

- Докажите, что для любого скошенного влево дерева $|T| \geq 2^{\text{rank}(T)} - 1$ ($|T|$ обозначает количество вершин в дереве T).
 - Придумайте, как слить две скошенные влево кучи H_1 и H_2 за время $\mathcal{O}(\log |H_1| + \log |H_2|)$.
 - Придумайте, как используя операцию слияния, построенную на предыдущем шаге, реализовать операции:
 - $\text{Insert}(x)$ — добавление элемента x в кучу,
 - $\text{ExtractMin}()$ — удаление минимального элемента из кучи.
- Покажите, что n операций **Add** подряд в пустую биномиальную кучу работают за $\mathcal{O}(n)$.
 - Предъявите последовательность действий, при которой **LeftistHeap** из пустой станет бамбуком (цепочкой из вершин, у каждой один ребенок).

4.2 Домашнее задание

1. В свободное время Анка-пулемётчица любит сортировать патроны по серийным номерам. Вот и сейчас она только разложила патроны на столе в строго отсортированном порядке, как Иван Васильевич распахнул дверь с такой силой, что все патроны на столе подпрыгнули и немного перемешались. Оставив ценные указания, Иван Васильевич отправился восвояси. Как оказалось, патроны перемешались не сильно. Каждый патрон отклонился от своей позиции не более чем на k . Всего патронов n . Помогите Анке отсортировать патроны.

- Отсортируйте патроны за $\mathcal{O}(nk)$.
- Отсортируйте патроны за $\mathcal{O}(n + I)$, где I — число инверсий.
- Докажите нижнюю оценку на время сортировки $\Omega(n \log k)$.
- Отсортируйте патроны за $\mathcal{O}(n \log k)$.

2. Рассмотрим бинарную скошенную систему исчисления. На каждой позиции в скошенной записи числа может стоять цифра 0, 1 или 2. Число $\overline{a_k a_{k-1} \dots a_2 a_1}$ в скошенной системе переводится в десятичную по формуле $\sum_{i=1}^k a_i \cdot (2^i - 1)$.

В скошенной системе счисления есть два ограничения: цифра 2 может встречаться в записи не более одного раза; все цифры следующих меньших разрядов равны нулю. Пример первых чисел: 1, 2, 10, 11, 12, 20, 100, 101...

- Докажите, что каждое неотрицательное целое число имеет единственное возможное представление в скошенной системе счисления.
- Придумайте, как увеличить число в скошенной системе на единицу за $\mathcal{O}(1)$.

3. Определим структуру данных «скошенный список». Список длины n строится так:

- запишем число n в скошенной системе счисления: $\overline{a_k a_{k-1} \dots a_2 a_1}$
- для каждого i смотрим в соответствующую позицию скошенной записи числа n и создаём a_i полных двоичных деревьев высоты i
- размещаем n элементов списка: сперва выбираем дерево в порядке возрастания высоты, а внутри конкретного дерева размещаем в порядке обхода в глубину: «корень, левый ребёнок, правый ребёнок»

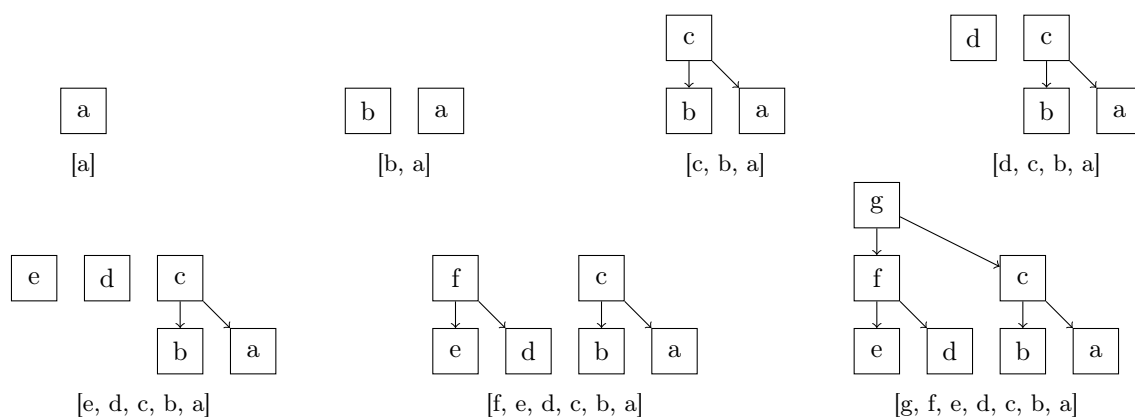


Рис. 1: Примеры скошенных списков

Придумайте, как реализовать следующие операции со списком длины n :

- Добавление элемента в начало списка за $\mathcal{O}(1)$.
- Доступ к i -му по порядку элементу за $\mathcal{O}(\log n)$.

(с) Получить скопированный список из k последних элементов данного скопированного списка за $\mathcal{O}(\log n)$.

4. Напишите псевдокод для структуры данных, реализующей операции:

- `push(key, priority)` — добавить ключ `key` (которого там не было) с приоритетом `priority`.
- `pop()` — извлечь ключ с минимальным приоритетом.
- `decrease_priority(key, new_priority)` — уменьшить приоритет ключа `key` (который в этот момент присутствует в структуре).

Все ключи — целые числа от 1 до n . Все операции должны работать за детерминированные $\mathcal{O}(\log n)$.

Дополнительные задачи

5. Куча хранится в массиве $h[0 \dots n-1]$ длины n . Родитель p хранит детей в ячейках $2 \cdot p + 1$ и $2 \cdot p + 2$. Алгоритм приступает к сортировке. Сортировка устроена следующим образом.

- Поменять первый и последний элемент кучи местами.
- Уменьшить n на единицу.
- Запустить `SiftDown` на первом элементе.

```
void SiftDown(int i) {
    int j = 2 * i + 1;
    if (j + 1 < n && h[j + 1] > h[j])
        j = j + 1;
    if (j < n && h[j] > h[i]) {
        swap(h[i], h[j]);
        SiftDown(j);
    }
}
```

Придумайте алгоритм, который по n выдаёт перестановку чисел от 1 до n , которая является корректной кучей и приводит к максимальному количеству вызовов `SiftDown` при сортировке. Время работы — $\mathcal{O}(n \log n)$.

5 Шустрая сортировка

5.1 Практика

1. Даны два массива a и b одинаковой длины. Найти такую перестановку p , что

(a) $\sum_{i=1}^n a_{p_i} b_i \rightarrow \max;$

(b) $\sum_{i=1}^n a_{p_i} b_i \rightarrow \min.$

2. Робот Иван Семеныч пробует пирожки. Содержимое пирожков делится на три типа. Всего пирожков n . Каждый пирожок можно попробовать не более одного раза. Любые два пирожка можно поменять местами. Память у робота маленькая, $\mathcal{O}(\log n)$ бит. Помогите Ивану Семенычу отсортировать пирожки по типу: сначала первый, потом второй, потом третий. Сортировка должна работать за линейное время.
3. Дана последовательность из n чисел, нужно за один проход и $\mathcal{O}(n)$ времени найти в ней k минимумов, используя $\mathcal{O}(k)$ дополнительной памяти.
4. Даны два массива из положительных чисел a и b , $|a| = |b| = n$. Выбрать массив p из k различных чисел от 1 до n так, чтобы $\frac{\sum_{i=1}^k a_{p_i}}{\sum_{i=1}^k b_{p_i}} \rightarrow \max$.
5. Есть длинный файл из n элементов, его можно 1 раз последовательно прочитать, но не хранить целиком. Требуется в конце вывести один из этих n элементов с равной вероятностью. n заранее неизвестно.
6. Алгоритм работает за $\mathcal{O}(n^2 \log n)$, вероятность успеха $\frac{1}{\log n}$. За какое время можно добиться вероятности успеха $1 - \frac{1}{n}$? Подсказка: $(1 - \frac{1}{x})^x \leq e^{-1}$ для $x \geq 1$.
7. Покажите, что не существует такой **вероятностной** сортировки сравнениями, которая корректно сортирует массив с вероятностью не менее $\frac{1}{2}$ и имеет матожидание времени работы в худшем случае $o(n \log n)$ (считайте, что сравнить пару элементов или сгенерировать 1 случайный бит занимает $\Theta(1)$ времени).
8. На прямой расположено n точек $p_1, p_2, \dots, p_n$ (p_i в произвольном порядке). Каждая точка имеет вес $w_i \geq 0$. Требуется найти точку q : $\sum_i [w_i \cdot |p_i - q|] \rightarrow \min$. $\mathcal{O}(n)$ в среднем.

Дополнительные задачи

9. Дан набор из n пар гаек и болтов, в разных парах размеры гаек и болтов различны. Гайки и болты перемешаны. Требуется для каждой гайки найти соответствующий болт. Сравнивать можно только болты с гайками (сравнить две гайки между собой, или два болта между собой — невозможно). $\mathcal{O}(n \log n)$.
10. Пусть в качестве разбивающего элемента всегда берется средний элемент: $\lfloor (l + r)/2 \rfloor$. Построить массив длины n , на котором QuickSort отработает за $\Omega(n^2)$.
11. Пусть мы хотим показать, что некоторая сортирующая сеть сливает два отсортированных массива в один. Покажите, что для этого достаточно показать, что она сливает массивы, состоящие из 0 и 1.

5.2 Домашнее задание

1. Пусть у нас есть массив $a[0..n-1]$ из **различных** чисел. Будем называть его *отсортированным на 90%*, если из него можно так вычеркнуть $\leq 10\%$ элементов, чтобы оставшиеся оказались в отсортированном порядке.

Рассмотрим следующий код:

```
def test_index(a, i):
    l = 0
    r = len(a)
    while r - l > 1:
        m = (l + r) // 2
        if a[m] <= a[i]:
            l = m
        else:
            r = m
    return (l == i)

def test_sort(a, k):
    for r in range(0, k):
        i = random.randrange(0, n)
        if not test_index(a, i):
            return False
    return True
```

- (a) Покажите, что, если массив отсортирован, то `test_sort` возвращает `True` с вероятностью 1.
 - (b) Покажите, что, если $i < j$ и `test_index(a, i)` и `test_index(a, j)` возвращают `True`, то $a[i] < a[j]$.
 - (c) Покажите, что, если **a не** отсортирован на 90%, то `test_sort(a, 1)` возвращает `True` с вероятностью $< \frac{9}{10}$.
 - (d) Подберите k (меньше — лучше), чтобы `test_sort(a, k)` возвращал `False` с вероятностью хотя бы $\frac{2}{3}$, если **a не** отсортирован на 90%.
2. Дан набор строчек s_i , состоящих из десятичных цифр, суммарной длины n . Найдите в каком порядке их нужно сконкатенировать, чтобы получившееся в результате число было максимальным. $\mathcal{O}(n^2 \log n)$.
 3. Дан массив из $2 \cdot n - 1$ числа, который нельзя модифицировать. Есть дополнительная память на $n + 1$ элемент массива и ещё $\mathcal{O}(1)$ сверху. Требуется найти медиану за $\mathcal{O}(n \log n)$.
 4. Даны массив из n чисел и m чисел $p_1, p_2, \dots, p_m$, нужно за $\mathcal{O}(n \log m + m)$ для каждого i найти p_i -ую порядковую статистику.
 5. Дан массив $A[1..n]$ из n различных чисел. Массив не обязательно отсортирован. Требуется найти k ближайших к медиане элементов за линейное в среднем время. Решить для двух метрик.

- (a) По позиции в отсортированном массиве.

$$d(x, \text{median}) = |\text{pos}(x) - \text{pos}(\text{median})|,$$

где $\text{pos}(x)$ — позиция элемента x в отсортированном массиве.

- (b) По значению.

$$d(x, \text{median}) = |x - \text{median}|.$$

6 Сортировки и порядковые статистики

6.1 Практика

1. Работает ли аналогичная оценка времени работы детерминированного алгоритма поиска порядковой статистики, если вместо пятерок разбивать элементы на
 - (a) семерки.
 - (b) тройки.
2. Придумайте, как добиться от QuickSort времени $\mathcal{O}(n \log n)$ в худшем случае.
3. Дан массив из n чисел от 1 до k , разработайте структуру данных, которая за $\mathcal{O}(1)$ отвечает на online запросы вида «Сколько в массиве элементов от a до b ?». Время на предподсчет $\mathcal{O}(n + k)$.
4. Даны n точек на плоскости (x_i, y_i) . Найти точку (x^*, y^*) :
 - (a) $\max_i \left[\max(|x_i - x^*|, |y_i - y^*|) \right] \rightarrow \min$
 - (b) $\max_i \left[|x_i - x^*| + |y_i - y^*| \right] \rightarrow \min$
 - (c) $\sum_i \left[(x_i - x^*)^2 + (y_i - y^*)^2 \right] \rightarrow \min$
 - (d) $\sum_i \left[|x_i - x^*| + |y_i - y^*| \right] \rightarrow \min$
 - (e) $\sum_i \left[\max(|x_i - x^*|, |y_i - y^*|) \right] \rightarrow \min$
5. Как с помощью цифровой сортировки сортировать строки над константным алфавитом в лексикографическом порядке за $\mathcal{O}(\sum |s_i|)$?
 - (a) Все строчки одинаковой длины.
 - (b) Строчки разной длины.
6. Возьмем массив a из n элементов, каждый из которых — это число 1 до n . Циклический сдвиг номер i — это последовательность $a_i, a_{i+1}, \dots, a_{n-1}, a_0, a_1, \dots, a_{i-1}$.
 - (a) Предложите алгоритм сортировки циклических сдвигов в лексикографическом порядке за время $\mathcal{O}(n^2)$.
 - (b) Предложите алгоритм сортировки циклических сдвигов за $\mathcal{O}(n \log n)$.
7. На прямой расположено n точек $p_1, p_2, \dots, p_n$. Каждая точка имеет вес $w_i \geq 0$. Выбрать две точки q_1, q_2 : $\sum_i w_i \min(|p_i - q_1|, |p_i - q_2|) \rightarrow \min$.

Дополнительные задачи

8. Пусть мы хотим показать, что некоторая сортирующая сеть сливает два отсортированных массива в один. Покажите, что для этого достаточно показать, что она сливает массивы, состоящие из 0 и 1.

6.2 Домашнее задание

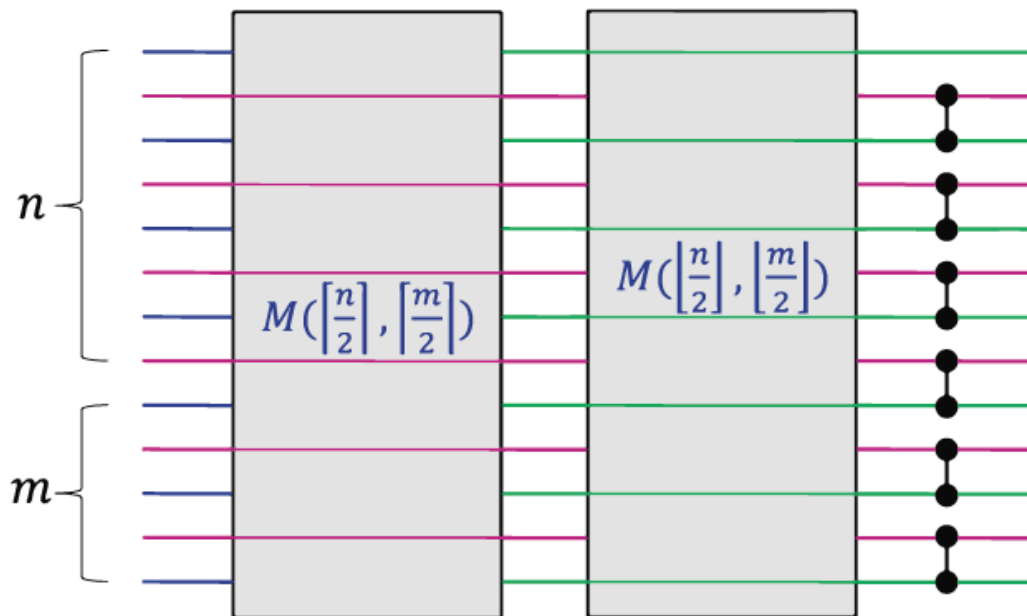
1. Есть улица длины l , которая освещается $n > 0$ фонарями, i -й фонарь находится в точке с вещественной координатой a_i . Фонарь освещает все точки улицы, которые находятся от него на расстоянии не больше d , где d — некоторое положительное число, общее для всех фонарей. Найдите минимальное d , при котором вся улица освещена. $\mathcal{O}(n)$, a_i не отсортированы.
2. Есть массив a_i , состоящий из n w -битных машинных слов, которые интерпретируются как неотрицательные целые числа. $n < 2^w - 1$. Найдите минимальное неотрицательное целое число, которого нет в массиве, за $\mathcal{O}(n)$ времени и $\mathcal{O}(1)$ машинных слов дополнительной памяти. Входной массив доступен для записи.
3. Даны n точек на плоскости (x_i, y_i) с весами $w_i \geq 0$. Найти точку (x^*, y^*) :

$$\max_i \left[w_i (|x_i - x^*| + |y_i - y^*|) \right] \rightarrow \min$$

4. В матрице Q из натуральных чисел размера $N \times N$ найти подматрицу размера $H \times W$ с максимальной медианой. H, W — нечётные.
 - (a) $\mathcal{O}(N^2 \log Q_{\max})$. Здесь $Q_{\max}$ — максимальный элемент матрицы.
 - (b) $\mathcal{O}(N^2 \log N)$.
5. Дан набор непустых строк суммарной длины n над алфавитом A . Отсортируйте строки за $\mathcal{O}(n + |A|)$.

Дополнительные задачи

6. Будем строить сортирующую сеть $M(n, m)$ для слияния двух отсортированных массивов размера n и m следующим образом. Разделим элементы обоих массивов на четные и нечетные, рекурсивно сольем отдельно четные, отдельно нечетные, в конце добавим компараторы для элементов $2k$ и $2k + 1$ (см картинку):



- (a) Покажите, что такая сеть действительно сливает два отсортированных массива.
- (b) Каково будет общее число компараторов в такой сети?
- (c) Какова будет глубина такой сети?

7 Демоническое программирование

7.1 Практика

1. Даны две последовательности длины n . Придумайте, как найти самую длинную общую подпоследовательность этих последовательностей.
 - (a) За $\mathcal{O}(n^2)$.
 - (b) За $\mathcal{O}(n \log n)$, если в одной из последовательностей все элементы различны, и вы умеете находить максимальную возрастающую подпоследовательность за $\mathcal{O}(n \log n)$.
2. Даны n предметов. У каждого заданы целый вес w_i и цена v_i . Найти $\max$ цену, которую можно набрать предметами суммарного веса $\leq S$. Время $\mathcal{O}(nS)$, память $\mathcal{O}(S)$.
 - (a) Каждый предмет можно брать сколько угодно раз.
 - (b) Каждый предмет можно брать один раз.
 - (c) Каждый предмет можно брать сколько угодно раз. Восстановить ответ.
 - (d) Каждый предмет можно брать один раз + набрать ровно S . Цен нет. Восстановить ответ.
 - (e) (*) Каждый предмет можно брать один раз + набрать ровно S . Восстановить ответ.
3. Дано рекуррентное соотношение:

(a)

$$\begin{aligned}a_n &= a_{n-1} + 2b_{n-1} \\ b_n &= 10b_{n-1} - a_{n-1}\end{aligned}$$

(b)

$$\begin{aligned}a_n &= a_{n-1} + 2c_{n-1} + 1 \\ b_n &= 5 - c_{n-1} \\ c_n &= c_{n-2} - b_{n-1}\end{aligned}$$

Известны $a_0, a_1, b_0, b_1, c_0, c_1$. Найти a_n, b_n, c_n по модулю p за $\mathcal{O}(\log n)$.

4. Дан выпуклый n -угольник. Каждая диагональ n -угольника, соединяющая вершины i и j , имеет вес w_{ij} . Вес триангуляции многоугольника есть сумма весов диагоналей, которые в ней проведены. Найти триангуляцию с минимальным весом за $\mathcal{O}(n^3)$.

Триангуляция — набор взаимно непересекающихся (кроме, возможно, концов) диагоналей, разбивающий многоугольник на треугольники.
5. Судно атакуют пираты. Для каждого пирата известны его азимут a_i и время t_i , через которое пират приплывет и совершит непотребство. Однако, у судна есть лазерная пушка, которой оно защищается. У пушки есть начальный азимут a и угловая скорость вращения ω . Пушка уничтожает все объекты, на которые она сейчас направлена. Помогите судну определить порядок уничтожения пиратов за $\mathcal{O}(n^2)$, чтобы не допустить непотребства.
6. Шаблоном будем называть строку, состоящую из букв латинского алфавита и символов “?” и “*”. Будем говорить, что строка s подходит под шаблон p , если в p можно заменить каждый символ ? на букву и каждый символ * на строку из латинских букв (возможно, пустую) так, что результат будет равен s . Для строки s и шаблона p определите, подходит ли строка под шаблон за $\mathcal{O}(|s| \cdot |p|)$. Подсчитайте количество способов, которыми строка подходит под шаблон.

Дополнительные задачи

7. Даны два шаблона a и b , состоящие из букв латинского алфавита и символов “?” и “*”. Найдите какое-либо слово минимальной длины, подходящее под оба шаблона, за время $\mathcal{O}(|a| \cdot |b|)$.

7.2 Домашнее задание

1. Дан набор нечестных монеток с вероятностью выпадения орла $p_1, p_2, \dots, p_n$. Требуется посчитать вероятность выпадения ровно k орлов за $\mathcal{O}(n \cdot k)$. Операции над числами считать выполнимыми за $\mathcal{O}(1)$.
2. Дана строка s длины n . Для каждой пары (i, j) найти длину максимального общего префикса i -го и j -го суффиксов строки s . $\mathcal{O}(n^2)$.

3. LZSS

Дана строка из латинских букв длины n , нужно ее запаковать в максимально короткую, используя правило (k, i) — повторить k символов начиная с i -й позиции (i -я позиция должна быть левее текущей, но отрезок $[i, i + k - 1]$ может покрывать текущую позицию).

Заметим, что длина (k, i) — не константа. Например, $xyababababz \rightarrow xyab(8, 2)z$, $xyaaaaabaaaaab \rightarrow xya(3, 2)b(10, 2)$ (но это не оптимально, оптимально $xyaaaaab(10, 2)$).

- (a) $\mathcal{O}(n^3)$.
- (b) $\mathcal{O}(n^2)$, считая, что длина строки (k, i) — константа.
- (c) $\mathcal{O}(n^2)$.

Подсказка: пусть $d[i]$ — длина самого короткого кода для суффикса строки начинающегося с i -й позиции...

4. (a) Даны пара чисел k, v и массив a размера W . Постройте за $\mathcal{O}(W)$ массив b такого же размера, где $b_i = \max_{0 \leq j \leq \min(k, i)} (a_{i-j} + v \cdot j)$.

Подсказка: $a_{i-j} + vj = a_{i-j} - v(i-j) + vi$.

- (b) Даны n предметов. У каждого есть цена v_i и вес w_i . Каждый предмет можно взять от 0 до $k_i > 0$ раз. Найдите за $\mathcal{O}(nW)$ максимальную суммарную цену, которую можно набрать таким образом, чтобы суммарный вес не превышал W .

Подсказка: придумайте, как пересчитать динамику для каждого остатка по модулю w_i .

5. Пусть есть n подарков разной натуральной стоимости и три поросёнка. Нужно раздать подарки как можно честнее (так, чтобы минимизировать разность суммарной стоимости подарков самого везучего поросёнка и самого невезучего). Придумайте алгоритм решения данной задачи за $\mathcal{O}(nW^2)$, где W — суммарная стоимость подарков.

8 Динамическое программирование #2

8.1 Практика

1. Дан выпуклый n -угольник. Каждая диагональ n -угольника, соединяющая вершины i и j , имеет вес w_{ij} . Вес триангуляции многоугольника есть сумма весов диагоналей, которые в ней проведены. Найти триангуляцию с минимальным весом за $\mathcal{O}(n^3)$.
Триангуляция — набор взаимно непересекающихся (кроме, возможно, концов) диагоналей, разбивающий многоугольник на треугольники.
2. Судно атакуют пираты. Для каждого пирата известны его азимут a_i и время t_i , через которое пират приплывет и совершит непотребство. Однако, у судна есть лазерная пушка, которой оно защищается. У пушки есть начальный азимут a и угловая скорость вращения ω . Пушка уничтожает все объекты, на которые она сейчас направлена. Помогите судну определить порядок уничтожения пиратов за $\mathcal{O}(n^2)$, чтобы не допустить непотребства.
3. Найти максимальное по весу паросочетание за $\mathcal{O}(n)$ на
 - (a) дереве из n вершин,
 - (b) простом цикле из n вершин,
 - (c) связном неориентированном графе из n вершин и n рёбер.Веса на рёбрах.
4. Дано w -битное машинное слово.
 - (a) Проверить, является ли слово степенью двойки. $\mathcal{O}(1)$.
 - (b) Проверить, правда ли в битовой записи никакие две единицы не идут подряд. $\mathcal{O}(1)$.
 - (c) Найти номер младшего единичного бита. $\mathcal{O}(\log w)$.
 - (d) Найти номер старшего единичного бита. $\mathcal{O}(\log w)$.
 - (e) Найти маску младшего единичного бита. $\mathcal{O}(1)$.
 - (f) Найти чётность количества бит. $\mathcal{O}(\log w)$.
 - (g) (*) Найти количество единичных бит. $\mathcal{O}(\log w)$.
5. Пусть дан оргграф на n вершинах, $n \leq w$ (w — разрядность машинного слова).
 - (a) На ребрах есть веса. Найдите кратчайший гамильтонов путь за $\mathcal{O}(n^2 2^n)$.
 - (b) На ребрах есть веса. Найдите кратчайший гамильтонов цикл за $\mathcal{O}(n^2 2^n)$.
 - (c) Проверьте существование гамильтонова пути за $\mathcal{O}(n 2^n)$.
 - (d) Посчитайте количество простых циклов за $\mathcal{O}(n^2 2^n)$.
6. Вычислите, сколькими способами можно замостить доминошками клетчатое поле
 - (a) $n \times 3$, за время $\mathcal{O}(n)$.
 - (b) $n \times 3$, за время $\mathcal{O}(\log n)$.
 - (c) $n \times m$, за время $\mathcal{O}(4^m n)$.
 - (d) $n \times m$, за время $\mathcal{O}(2^m nm)$.

Ответ посчитать по модулю небольшого простого числа.

Дополнительные задачи

7. Есть k грузовиков с заданной вместимостью, задача — перевезти n вещей с заданными весами минимальным числом заездов. Один заезд — погрузить и отправить все грузовики.
 - (a) $k = 1$, $\mathcal{O}(3^n)$.
 - (b) $k = 2$, $\mathcal{O}(4^n)$.
 - (c) $\mathcal{O}(3^k n)$.

8.2 Домашнее задание

1. Дан циклический массив, перед i -м идёт элемент $(i-1) \bmod n$, после i -го идёт $(i+1) \bmod n$. Рассмотрим операцию `zero(i) { ans += a[next(i)]*a[i]*a[prev(i)]; a[i] = 0 }`. Найти последовательность операций, максимизирующую `ans`.
(a) $\mathcal{O}(n^3)$.
(b) $\mathcal{O}(n)$.
2. В Галлии есть n деревень, некоторые из которых соединены дорогами, длины которых – натуральные числа. Известно, что от каждой деревни до любой другой можно добраться единственным способом. Астерикс и Обеликс решили устроить праздничный забег в честь очередной победы над подразделением Цезаря. Они хотят, чтобы забег начинался и заканчивался в деревнях, маршрут не проходил ни по какой деревне дважды и имел длину ровно m . Помогите им посчитать количество различных возможных маршрутов за $\mathcal{O}(nm)$.
3. Дана строка из латинских букв длины n , нужно ее за $\mathcal{O}(n^3)$ запаковать в максимально короткую, используя правило $n(S) = \underbrace{SS \dots S}_n$.
Например `UNIXYESYESYESUNIXYESYESYES` $\rightarrow$ `2(UNIX3(YES))`.
4. Пусть даны множества $B, A_1, A_2, \dots, A_m \subset \{0, 1, \dots, n-1\}$. Представьте B в виде объединения минимального количества A_k . $\mathcal{O}(m2^n)$.
5. Найдите во взвешенном графе гамильтонов цикл минимального веса, который удовлетворяет дополнительно следующему свойству: сначала номера посещенных вершин возрастают, а затем убывают. Время $\mathcal{O}(n^2)$.

Дополнительные задачи

6. Есть мешок с p камнями с массами w_i . Суммарный вес камней равен W . Известно, что если столкнуть два камня i, j с весами $w_i \geq w_j$, то они уничтожатся, а на их месте останется камень-осколок весом $w_i - w_j$. Разрешается в произвольном порядке сталкивать по два камня, возвращая результаты столкновений обратно в мешок, до тех пор, пока останется не более одного камня. Определите минимальный возможный вес мешка в конце. $\mathcal{O}(nW)$.
7. Найти максимальное по весу паросочетание на кактусе за $\mathcal{O}(n)$. Кактус — граф, в котором каждое ребро лежит не более чем на одном простом цикле.
8. Есть n вещей, у каждой есть стоимость v_i и вес w_i . Есть рюкзак, в котором можно унести набор вещей суммарного веса не более W за один подход. За $m = 2^k$ подходов унести вещи максимальной суммарной стоимости. Время $\mathcal{O}(3^n k)$.
9. Вам дана доска фанеры размера $n \times m$. В нее было вбито несколько гвоздей с целыми координатами (от них остались некрасивые дырки). Сколькими способами можно разрезать доску на прямоугольники с целыми сторонами так, чтобы ни один из гвоздей не попал внутрь прямоугольника. Время: $\mathcal{O}(n^2 4^m)$.
10. Найти минимальный шаблон такой, что под него не подходит первая строка, но при этом подходит вторая. $\mathcal{O}(n^3)$.

9 Жадные алгоритмы

9.1 Практика

1. Постройте код Хаффмана за $\mathcal{O}(n)$, если частоты букв уже даны в отсортированном порядке.
2. В фирму поступило n заказов, которые можно выполнять в произвольном порядке. На выполнение заказа i необходимо время t_i . В каждый момент времени можно работать ровно над одним заказом. Пусть e_i — момент окончания выполнения заказа номер i . Распределите работу над заказами так, чтобы минимизировать $\sum_i e_i$. $\mathcal{O}(n \log n)$.
3. Есть n работ, у каждой есть $t_i > 0$ — время ее выполнения и d_i — дедлайн.
 - (a) Определите, можно ли выполнить их все. $\mathcal{O}(n \log n)$.
 - (b) Найдите максимальное подмножество, которое можно выполнить. $\mathcal{O}(n \log n)$.
4. Даны n гномов. Если i -го гнома укладывать спать a_i минут, он потом спит b_i минут. Можно ли сделать так, чтобы в какой-то момент все гномы спали? $\mathcal{O}(n \log n)$.
5. Даны n монеток, у каждой есть своя вероятность выпадения орла p_i . Нужно выбрать подмножество размера k из них, для которого вероятность выпадения ровно $\frac{k}{2}$ орлов при одно-временном подбрасывании максимальна (k — четное).
 - (a) $\mathcal{O}(n \log n + k^3)$.
 - (b) $\mathcal{O}(n + k^2)$.
6. Машина тратит единицу топлива на километр, имеет бак объема k и находится в начале прямой дороги в точке 0. Для всех $i \in \mathbb{N} \cup \{0\}$ в i километрах от неё есть заправочная станция со своей положительной ценой c_i . Определите за время $\mathcal{O}(n)$, как проехать n километров за минимальную стоимость.
7. Маршрутка совершает рейс от первой до n -й остановки. В маршрутке m мест для пассажиров. Есть k человек, про каждого заранее известно, что он хочет доехать от остановки s_i до f_i . Проезд для пассажира стоит 1 вне зависимости от расстояния между остановками. Максимизируйте прибыль, при условии, что можно выбирать, кого посадить в маршрутку на каждой остановке. $\mathcal{O}((n + m + k) \log m)$.

Дополнительные задачи

8. Будем называть *независимым множеством* (или *антикликой*) подмножество вершин графа, в котором нет ребер. Пусть в графе G есть V вершин и E ребер, а максимальная степень равна d . Найдите в нём независимое множество размера хотя бы:
 - (a) $\frac{n}{d+1}$ за время $\mathcal{O}(V + E)$
 - (b) $\sum_{v \in V(G)} \frac{1}{\deg(v)+1}$ за время $\mathcal{O}(V \log V + E)$
- Считайте, что граф уже дан в памяти в виде массива, где для каждой вершины хранится список её соседей.
9. Имеется n деталей и два станка. Каждая деталь должна сначала пройти обработку на первом станке, затем — на втором. При этом i -ая деталь обрабатывается на первом станке за a_i времени, а на втором — за b_i времени. Каждый станок в каждый момент времени может работать только с одной деталью. Требуется составить такой порядок подачи деталей на станки, чтобы итоговое время обработки всех деталей было бы минимальным. $\mathcal{O}(n \log n)$.

9.2 Домашнее задание

1. (а) Пусть в тексте встречается символ с частотой большей $\frac{2}{5}$. Докажите, что **любой** двоичный беспрефиксный код, который минимизирует длину закодированного текста, будет содержать слово длины 1.
Подсказка: любой — значит не только тот, который получается алгоритмом Хаффмана.
(б) Пусть в тексте частоты всех символов меньше $\frac{1}{3}$. Докажите, что **любой** двоичный беспрефиксный код, который минимизирует длину закодированного текста, содержит только кодовые слова длины не менее 2.
2. Преподаватели сделали n заявок на занятие. Каждое занятие начинается в момент b_i и кончается в момент e_i (занимает интервал $[b_i, e_i]$). Два занятия в одной аудитории быть не могут. Распределите заявки по аудиториям так, чтобы общее число аудиторий было минимально. Решить за $\mathcal{O}(n \log n)$.
3. n атлетов хотят выстроить из своих тел башню максимальной высоты. Башня это цепочка атлетов, первый стоит на земле, второй стоит у него на плечах, третий стоит на плечах у второго и т.д. Каждый атлет характеризуется силой s_i и массой m_i . Сила это максимальная масса, которую атлет способен держать у себя на плечах. Известно, что если атлет тяжелее, то он и сильнее, но атлеты равной массы могут иметь различную силу. Определите максимальную высоту башни. $\mathcal{O}(n \log n)$.
4. Есть n человек. Человек i готов примкнуть к нашей банде, если наш авторитет хотя бы a_i , при этом он изменит наш авторитет на b_i . Наш изначальный авторитет равен A . $a_i, b_i, A \in \mathbb{Z}$
(а) Можем ли завербовать всех людей? $\mathcal{O}(n \log n)$.
(б) Какое максимальное число людей мы можем завербовать? $\mathcal{O}(n \log n)$.

Подсказка: попробуйте сначала решить ограниченные варианты задачи с $b_i \geq 0$ и с $b_i < 0$.

Дополнительные задачи

5. **Пятачок, у тебя есть дома ружье?**
Даны n непересекающихся кругов на плоскости. Мы стоим в точке $(0, 0)$ и можем стрелять по прямой. Минимальным числом выстрелов проткнуть все круги.
6. n школьников упали в яму глубины S . Каждый школьник имеет рост (от ног до плеч) h_i и длину рук l_i . Школьники могут вставать друг другу на плечи, верхний школьник может вытянуть руки. Чтобы выбраться из ямы необходимо дотянуться руками до уровня земли.
(а) Могут ли выбраться все школьники? $\mathcal{O}(n \log n)$.
(б) Какое максимальное число школьников может выбраться? $\mathcal{O}(n \log n)$.
7. В фирму поступают заказы, которые можно выполнять в произвольном порядке. В каждый момент времени можно работать ровно над одним заказом. Изначально заказов нет, i -й заказ поступает в момент времени r_i , работать над ним нужно t_i . Пусть e_i — момент окончания выполнения заказа номер i . Распределите работу над заказами так, чтобы минимизировать $\sum_i e_i$. Переходить от одного заказа к другому можно в любой момент времени (даже если заказ не доделан до конца, незаконченный заказ можно будет возобновить с того же места). Свойства заказа (r_i, t_i) не известны до момента его поступления. Время $\mathcal{O}(n \log n)$, при условии, что всего поступит n заказов.
8. *Раскраской вершин* графа $G = (V, E)$ называется функция $c : V \rightarrow [m]$, сопоставляющая каждой вершине G цвет от 1 до m . Раскраска называется *правильной*, если каждая пара соседних вершин имеет разные цвета.
Для неориентированного графа $G = (V, E)$ его *хроматическим числом* $\chi(G)$ называется наименьшее возможное число цветов в правильной раскраске G .

Для графа G обозначим размер его максимального полного подграфа через $\omega(G)$.

Рассмотрим на вещественной прямой замкнутые отрезки $I_1, I_2, \dots, I_n$. Сопоставим каждому отрезку I_i вершину v_i и каждой паре пересекающихся отрезков (I_i, I_j) ребро (v_i, v_j) . Такой граф будем называть *интервальным графом*.

Будем называть граф G *совершенным*, если для любого его индуцированного подграфа H верно $\omega(H) = \chi(H)$.

Докажите, что каждый интервальный граф совершенен. Приведите алгоритм, красящий интервальный граф $G = (V, E)$ с $|V| = n$ в $\omega(H)$ цветов за время $O(n \log(n))$.

9. В фирму поступают заказы, которые можно выполнять в произвольном порядке. В каждый момент времени можно работать ровно над одним заказом. Изначально заказов нет, i -й заказ поступает в момент времени r_i , работать над ним нужно t_i .

Все заказы объединены в проекты (один заказ относится к одному проекту, заказы из одного проекта могут поступать не подряд).

Пусть e_i – момент окончания выполнения последнего (в порядке выполнения) из заказов в проекте с номером i .

Нужно распределить работу над заказами так, чтобы минимизировать $\sum_i e_i$. Свойства заказа $(r_i, t_i, \text{его проект})$ не известны до момента его поступления. Переходить от одного заказа к другому можно в любой момент времени (даже если заказ не доделан до конца).

Придумайте решение, которое не более чем в два раза хуже оптимального. Время $\mathcal{O}(n \log n)$, при условии, что всего поступит n заказов.

10 DSU и остовные деревья

10.1 Практика

1. Дан набор вершин, в него добавляются ребра; **online**, после каждого добавления говорить, является ли граф двудольным. $\mathcal{O}(m \log^* n)$.
2. В каждой клетке прямой записано число 0 или 1. Поступает информация: четность числа единиц на отрезке $[L_i, R_i]$, найти первый запрос, после которого данные противоречивы. $\mathcal{O}(m \log^* n)$.
3. В стране n аэропортов. Самолет может сделать перелет из аэропорта i в аэропорт j , израсходовав w_{ij} горючего. При этом $w_{ij} = w_{ji}$, и $w_{ii} = 0$. Требуется найти минимальный размер бака, позволяющий добраться самолету из любого города в любой, возможно с дозаправками. Решить за $\mathcal{O}(n^2)$.
4. Пусть даны взвешенный неорграф с неединственным минимальным остовным деревом и какой-то из его минимальных остовов. Найдите минимальный остов графа, отличный от данного. $\mathcal{O}(E \log V)$.
5. Найдите во взвешенном неорграфе такой цикл, что максимальный вес ребра этого цикла минимален. $\mathcal{O}((V + E) \log E)$.
6. Дан набор из n корневых деревьев, каждое из одной вершины. Операции **online**:
 - подвесить за корень одно из деревьев к другому ребром заданного веса,
 - посчитать сумму весов ребер на пути из вершины a в ее предка b (гарантируется, что запросы поступают только про пары вершина-предок).

(a) $\mathcal{O}(\log n)$ на операцию.

(b) $\mathcal{O}(\log^* n)$ на операцию.
7. Бюрократы ставят печати. В компании есть n бюрократов. Сверху спустили поручение выполнить m заданий в заданном порядке. Задания могут быть трех типов.
 - Назначить x начальником y (гарантируется, что у y до этого начальства не было).
 - Поручить x проставить печать на документе i , где i – номер запроса. После этого x передает документ начальнику, тот ставит печать, передает начальнику и т.д. Самый главный начальник ставит печать и сканирует документ шредеру.
 - Узнать, ставил ли x печать на документе i .

Поскольку все документы порезаны на мелкие кусочки, на вас последняя надежда. Нужно ответить на все запросы за $\mathcal{O}(m \log^*(n))$ **offline**.
8. Дан взвешенный связный неориентированный граф $G = \langle V, E \rangle$ и некоторое минимальное остовное дерево на нём. Пусть для некоторого ребра $e \in E$ изменился вес. По графу, остовному дереву, ребру e и его новому весу найдите новое минимальное остовное дерево за $\mathcal{O}(V + E)$.

Дополнительные задачи

9. Рассмотрим следующий алгоритм поиска минимального покрывающего дерева (алгоритм Борушки):
 - Пока в графе больше одной вершины:
 - Для каждой вершины найдем самое легкое инцидентное ей ребро и добавим его в множество S (одно и то же ребро может быть выбрано дважды).
 - Добавим все ребра из множества S в ответ.
 - Стынем граф по ребрам из S .

Докажите, что такой алгоритм найдет минимальное покрывающее дерево в случае, если веса всех ребер в графе различны, при этом время работы будет $\mathcal{O}(E \log V)$.

Придумайте, как модифицировать алгоритм, если возможны равные веса.

10. (a) Постройте пример для алгоритма Борувки, на котором он делает $\Theta(\log n)$ фаз.
(b) Постройте пример для алгоритма Борувки, на котором он делает $\Theta(1)$ фаз.
11. Дан взвешенный орграф, постройте ориентированное к корню остовное дерево с корнем в вершине 1 минимального веса.

10.2 Домашнее задание

1. Дан неориентированный граф с весами на вершинах, в котором в начальный момент нет ребер. Вершин V штук. Нужно обрабатывать следующие запросы:

- соединить ребром пару вершин.
- по вершине узнать размер связной компоненты, которой вершина принадлежит, и самую легкую вершину в этой компоненте.

$\mathcal{O}(\log^* V)$ на запрос в среднем, **online**.

2. Есть отрезок длины n и m запросов вида $paint(l, r, c)$ – перекрасить подотрезок $[l..r]$ в цвет c (все координаты целые). Определите состояние отрезка после выполнения всех запросов за $\mathcal{O}((n + m) \log^* n)$.

Подсказка: выполняйте запросы с конца и используйте идею из предыдущей задачи.

3. Дан неориентированный связный взвешенный граф $\langle V, E, w \rangle$. Надо для каждого ребра $e \in E$ установить, существует ли в графе такое минимальное остовное дерево T , что $e \in T$. $\mathcal{O}(|E| \log |V|)$.
4. Вам нужно передать с одного компьютера на другой n файлов. Файл i можно передать либо просто как последовательность битов, либо как **diff** с другим файлом j , уже пересланным ранее. В первом случае надо будет передать $m_i > 0$ бит, во втором — $d_{ij} = d_{ji} > 0$ бит. Найдите минимальное число битов, которое нужно передать, чтобы отправить все файлы с одного компьютера на другой. $\mathcal{O}(n^2)$.

11 DFS

11.1 Практика

1. Даны два множества вершин: A и B , за $\mathcal{O}(V + E)$ проверить, есть ли путь из какой-нибудь вершины $a \in A$ в какую-нибудь вершину $b \in B$.
2. Ориентировать неорграф так, чтобы он стал ациклическим за $\mathcal{O}(V + E)$.
3. Дано подвешенное дерево, нужно с линейным предпосчётом научиться отвечать на запрос “правда ли вершина u лежит в поддереве вершины v ” online за $\mathcal{O}(1)$.
4. Проверить, является ли данный неориентированный граф двудольным $\mathcal{O}(V + E)$.
5. Дан связный неорграф, нужно найти в нем за $\mathcal{O}(V + E)$ все
 - (a) мосты.
 - (b) точки сочленения.
6. Ориентировать неорграф так, чтобы он стал сильно связным за $\mathcal{O}(V + E)$.
7. Дан DAG,
 - (a) найти в нем гамильтонов путь за $\mathcal{O}(V + E)$.
 - (b) проверить единственность топологической сортировки за $\mathcal{O}(V + E)$.
8.
 - (a) Найти цикл в орграфе через данное ребро за $\mathcal{O}(E)$.
 - (b) Найти цикл в орграфе через данную вершину за $\mathcal{O}(E)$
 - (c) Найти цикл в неорграфе через данное ребро за $\mathcal{O}(E)$.
 - (d) Найти цикл в неорграфе через данную вершину за $\mathcal{O}(E)$.
 - (e) Найти в неорграфе какой-нибудь цикл за $\mathcal{O}(V)$.
9. В стране n аэропортов. Самолет может сделать перелет из аэропорта i в аэропорт j , израсходовав $w_{ij} > 0$ горючего. При этом w_{ij} может отличаться от w_{ji} , и $w_{ii} = 0$. Требуется найти минимальный размер бака, позволяющий добраться самолету из любого города в любой, возможно с дозаправками. Решить за $\mathcal{O}(n^2 \log n)$.

Дополнительные задачи

10. Найти в орграфе все вершины, через которые проходит любой путь $a \rightsquigarrow b$. $\mathcal{O}(V + E)$.
11. За $\mathcal{O}(V + E)$ найти в неорграфе какой-нибудь цикл
 - (a) нечетной длины.
 - (b) четной длины.
12. У каждой вершины не более 3-х врагов. Разбить на 2 доли так, чтобы с вершиной в долю попало не более 1 врага. $\mathcal{O}(V + E)$.

11.2 Домашнее задание

1. Пусть дано дерево $T = \langle V, E \rangle$. Для каждого ребра вычислить, сколько простых путей проходит через него. $\mathcal{O}(|V| + |E|)$.
2. Корневое дерево T на V вершинах задается массивом из V элементов. Все вершины пронумерованы. Для каждой вершины в массиве указан её родитель. Для корня r значение в массиве равно -1. Требуется определить, как будет выглядеть новое представление дерева, если корень r сменить на корень q . Разрешается использовать $\mathcal{O}(1)$ дополнительной памяти. Менять массив можно. Время $\mathcal{O}(V)$.
3. За $\mathcal{O}(|V| + |E|)$ найдите в неорграфе:
 - (a) все **ребра**, через которые проходит любой путь $a \rightsquigarrow b$.
 - (b) все **вершины**, через которые проходит любой путь $a \rightsquigarrow b$.
4. Дано дерево $T = \langle V, E \rangle$.
 - (a) Найдите центроид дерева — вершину c , для которой минимальна $\sum_{v \in V} \text{dist}(v, c)$. $\mathcal{O}(|V|)$.
 - (b) Найдите 2-центроид дерева — пару вершин (a, b) , для которой минимальна $\sum_{v \in V} \min(\text{dist}(v, a), \text{dist}(v, b))$. $\mathcal{O}(|V|^2)$.

Дополнительные задачи

5. Для заданного ориентированного графа $G = \langle V, E \rangle$ посчитать минимальное число ребер, которые нужно добавить в граф, чтобы он стал сильно связным. Время $\mathcal{O}(|V|^3)$.
6. Дано дерево $T = \langle V, E \rangle$. Найдите 2-центроид за $\mathcal{O}(|V|)$.

12 BFS

12.1 Практика

1. Даны три бочки на c_1 , c_2 и c_3 литров ($c_1, c_2, c_3 \in \mathbb{N}$, $c_1 \leq c_2 \leq c_3$). Бочки с ёмкостями c_1 и c_2 заполнены водой. Разрешается переливать воду между бочками, либо пока одна бочка не заполнится полностью, либо пока другая не опустеет. Проверить, возможно ли получить 2 литра в любой из бочек, и, если возможно, найти минимальное необходимое число переливаний. $\mathcal{O}(c_1 \cdot c_2)$.
2. Найти количество путей (необязательно простых) в графе за $\mathcal{O}(V^3 \log k)$
 - (a) между всеми парами вершин длины ровно k .
 - (b) между парой вершин длины $\leq k$.
3. Найти в орграфе все вершины, через которые проходит какой-нибудь кратчайший путь $a \rightsquigarrow b$. $\mathcal{O}(V + E)$.
4. Пусть в графе есть ребра веса 0 и 1. Придумайте, как найти кратчайшее расстояние от вершины s до остальных за $\mathcal{O}(V + E)$.
5. Пусть в графе все ребра имеют целый вес из $[1, k]$. Придумайте, как найти кратчайшее расстояние от вершины s до остальных.
 - (a) $k = 2$, $\mathcal{O}(V + E)$
 - (b) k — произвольное, $\mathcal{O}(V + kE)$.
 - (c) k — произвольное, $\mathcal{O}(kV + E)$.
 - (d) k — произвольное, $\mathcal{O}(E \log k)$.
6. Дан орграф с положительными весами на ребрах. Найти кратчайший путь, проходящий по всем k выделенным вершинам. Время $\mathcal{O}(2^k E \log(2^k V))$.
7. В стране n аэропортов. Самолет может сделать перелет из аэропорта i в аэропорт j , израсходовав $w_{ij} > 0$ горючего. При этом w_{ij} может отличаться от w_{ji} , и $w_{ii} = 0$. Требуется найти минимальный размер бака, позволяющий добраться самолету из любого города в любой, возможно с дозаправками. Решить за $\mathcal{O}(n^2)$.
8.
 - (a) k -2k-bfs, веса вещественные.
 - (b) k -2k-bfs, веса целые, хотим делать только целочисленные операции.
9. Пусть длина пути определяется как сумма весов всех ребер по модулю n . Найти кратчайший путь за $\mathcal{O}((V + E)n)$.

Дополнительные задачи

10. Даны две параллельных прямых (река). В реке есть n островов (точек). Мы хотим провести по реке корабль, представляющий собой открытый круг радиуса R , так, чтобы он не задел ни одного острова. Найти максимальный R , при котором это еще возможно, за $\mathcal{O}(n^2)$.

12.2 Домашнее задание

1. Дан взвешенный орграф с положительными весами и вершина s . Нужно для каждой вершины v найти число кратчайших путей из s в v по модулю p . $\mathcal{O}(E \log V)$.
2. В орграфе есть бесплатные и K типов платных ребер. Передвижение по любому ребру стоит 0, но, чтобы двигаться по платному ребру типа x , необходимо иметь пропуск того же типа. В любой момент времени можно иметь не более одного пропуска, однако в любой вершине есть возможность купить пропуск любого типа за A и продать пропуск любого типа за B ($0 < B < A$, цены одинаковы во всех вершинах). Найдите самый дешевый способ добраться из вершины s до вершины t за $\mathcal{O}(K(V + E))$.
3. Дан ориентированный граф с выделенными вершинами s и t . Перемещение по любому ребру занимает 1 единицу времени. В начальный момент времени мы стоим в вершине s , и во все ребра с постоянной скоростью L начинает поступать вода. У каждого ребра e есть емкость C_e — минимальное количество воды в этом ребре, при котором по нему больше нельзя двигаться (мы должны закончить движение по ребру не позднее момента достижения его емкости). Найдите максимальное L , при котором все еще существует возможность добраться от s до t за $\mathcal{O}((V + E) \log C_{\max})$.
4. Постройте матрицу $n \times n$, состоящую из клеток-стенок и пустых клеток, на которой при запуске BFS из какой-то клетки максимальный размер очереди будет $\omega(n)$. Ходить можно между клетками, смежными по стороне.

Дополнительные задачи

5. Попробуем научить алгоритм Дейкстры работать на графах с отрицательными весами: вынимаем из кучи вершину v с минимальным расстоянием и релаксируем все ребра из v . При этом вершины, до которых улучшилось расстояние, добавляются в кучу (или делается `DecreaseKey`, если вершина уже там).
 - (a) Покажите, что этот алгоритм корректно вычисляет кратчайшие пути, если нет циклов отрицательного веса.
 - (b) Придумайте пример без циклов отрицательного веса, на котором время работы экспоненциально.
6. Модифицируем условия задачи про бочки: есть n бочек, i -я бочка имеет объем c_i ($c_i > 0$) и в начальный момент содержит x_i воды ($0 \leq x_i \leq c_i$). Операция переливания определена как в задаче из практики. Придумайте такие n и вещественные c_i, x_i , чтобы соответствующий граф состояний, достижимых с помощью переливаний из начального, получился бесконечным.

13 В предыдущих сериях

13.1 Практика

1. Рассмотрим следующий алгоритм поиска минимального покрывающего дерева (алгоритм Борувки):
 - Пока в графе больше одной вершины:
 - Для каждой вершины найдем самое легкое инцидентное ей ребро и добавим его в множество S (одно и то же ребро может быть выбрано дважды).
 - Добавим все ребра из множества S в ответ.
 - Стынем граф по ребрам из S .

Докажите, что такой алгоритм найдет минимальное покрывающее дерево в случае, если веса всех ребер в графе различны, при этом время работы будет $\mathcal{O}(E \log V)$.

Придумайте, как модифицировать алгоритм, если возможны равные веса.
2. (a) Постройте пример для алгоритма Борувки, на котором он делает $\Theta(\log n)$ фаз.
(b) Постройте пример для алгоритма Борувки, на котором он делает $\Theta(1)$ фаз.
3. Пусть B_n (биномиальное дерево порядка n) определено следующим образом:
 - при $n = 0$ это дерево из одной вершины.
 - при $n > 0$ это B_{n-1} , которому первым ребенком подвешено еще одно B_{n-1} .
 - (a) Докажите, что B_n имеет высоту n .
 - (b) Докажите, что в B_n содержится ровно 2^n вершин.
 - (c) Определим биномиальную кучу как набор биномиальных деревьев, в котором нет двух деревьев одного порядка. Покажите, что для любого n существует биномиальная куча с n вершинами.
 - (d) Пусть на всех деревьях биномиальной кучи выполняется свойство кучи (min в голове). Придумайте **GetMin** за $\mathcal{O}(\log n)$.
 - (e) Придумайте **Merge** за $\mathcal{O}(\log n)$.
 - (f) Придумайте **Add** за $\mathcal{O}(\log n)$.
 - (g) Придумайте **ExtractMin** за $\mathcal{O}(\log n)$.
 - (h) Придумайте **DecreaseKey** по ссылке на узел. $\mathcal{O}(\log n)$.
 - (i) Придумайте **Delete** по ссылке на узел. $\mathcal{O}(\log n)$.
4. По неориентированному графу и его минимальному остовному дереву найдите второе остовное дерево (т.е. самое легкое остовное дерево из несовпадающих с данным в условии минимальным; деревья сравниваются как множества ребер) за время $\mathcal{O}(V^2 + E)$.

Дополнительные задачи

5. Найти в **орграфе** все вершины, через которые проходит любой путь $a \rightsquigarrow b$. $\mathcal{O}(V + E)$.
6. Дан взвешенный орграф, постройте ориентированное к корню остовное дерево с корнем в вершине 1 минимального веса.
7. (a) k -2k-bfs, веса вещественные.
(b) k -2k-bfs, веса целые, хотим делать только целочисленные операции.
8. Дано бинарное дерево: $Tree ::= Node(Tree, Tree) | Empty$ (эта запись означает, что дерево — это либо вершина с парой потомков-деревьев, либо особое значение $Empty$). Определим функцию **rank**(x) следующим образом:

- $\mathbf{rank}(\text{Empty}) = 0$
- $\mathbf{rank}(\text{Node}(\text{left}, \text{right})) = \min(\mathbf{rank}(\text{left}), \mathbf{rank}(\text{right})) + 1.$

Назовём бинарное дерево *скошенным влево (левацким)*, если для его вершин выполнено следующее свойство:

$$\forall_{x=\text{Node}(\text{left}, \text{right})} \mathbf{rank}(\text{left}) \geq \mathbf{rank}(\text{right}).$$

Скошенная влево (левацкая) куча — это скошенное влево дерево, в вершинах которого хранятся данные, для которых выполнено свойство кучи.

- Докажите, что для любого скошенного влево дерева $|T| \geq 2^{\mathbf{rank}(T)} - 1$ ($|T|$ обозначает количество вершин в дереве T).
 - Придумайте, как слить две скошенные влево кучи H_1 и H_2 за время $\mathcal{O}(\log |H_1| + \log |H_2|)$.
 - Придумайте, как используя операцию слияния, построенную на предыдущем шаге, реализовать операции:
 - $\text{Insert}(x)$ — добавление элемента x в кучу,
 - $\text{ExtractMin}()$ — удаление минимального элемента из кучи.
9. Предъявите последовательность действий, при которой `LeftistHeap` из пустой станет бамбуком (цепочкой из вершин, у каждой один ребенок).

13.2 Домашнее задание

1. Дано дерево $T = \langle V, E \rangle$. Найдите 2-центроид за $\mathcal{O}(|V|)$.
2. За $\mathcal{O}(V + E)$ найдите в неорграфе какой-нибудь цикл четной длины.
3. Дан неориентированный граф G . Посчитайте количество простых циклов длины 5. $\mathcal{O}(V^3)$.
4. Даны n строк, состоящих из круглых скобок. Строка s называется ППСП (почти правильная скобочная последовательность), если в конец s можно дописать несколько закрывающих скобок, чтобы s стала правильной. Данные строки можно склеивать в произвольном порядке.
 - (a) Можно ли построить ППСП, используя все строки? $\mathcal{O}(n \log n)$.
 - (b) Выберите максимально число строк, чтобы получилась ППСП. $\mathcal{O}(n \log n)$.
5. Дан орграф, посчитайте от начальной вершины до всех остальных кратчайшие расстояния относительно метрики “вес самого легкого ребра на пути”. $\mathcal{O}(V + E)$.
6. В плоскость по оси Ox беспощадно вбили в ряд n гвоздиков (координаты $(1, 0)$, $(2, 0)$... $(n, 0)$). Напротив них с таким же успехом, но с координатой $y = 1$ вбили еще n гвоздиков. Продолжая вандализм, взяли n ниточек и связали пары гвоздиков так, что ни один гвоздик не повязан двумя ниточками, и каждая пара состоит из верхнего и нижнего гвоздика. Получилось паросочетание. По паросочетанию найти максимальное множество ниточек, каждая пара в котором пересекается. Решить за $\mathcal{O}(n \log n)$.
7. Алиса и Базилио, чтобы скоротать время до нового клиента, играют в игру: Алиса загадывает число от 1 до N , а Базилио пытается его отгадать. Они договорились, что Базилио может задавать только вопросы вида «Принадлежит ли число заданному множеству X ?», на которые Алиса отвечает либо «да», либо «нет».

Алиса всегда играет так, что в серии из K последовательных загадываний каждое из чисел $1, 2, \dots, N$ встречается ровно n_i раз (хотя и в разном порядке). К сожалению, у Базилио короткая память, и он не может запомнить даже предыдущее число, загаданное Алисой, а писать он не умеет, хотя читать его научили. Минимизируйте количество вопросов, которое Базилио в худшем случае должен задать Алисе в каждой серии из K загадываний, чтобы угадать все числа.
8. На прямой расположено n точек $p_1, p_2, \dots, p_n$. Каждая точка имеет вес $w_i \geq 0$. Выбрать три точки q_1, q_2, q_3 : $\sum_i \left[w_i \cdot \min(|p_i - q_1|, |p_i - q_2|, |p_i - q_3|) \right] \rightarrow \min$.

Требуется решение за линию на полилогарифм.
9. Сколько (по модулю p) существует перестановок, у которых все длины циклов хотя бы k ?
 - (a) $\mathcal{O}(n^2)$.
 - (b) $\mathcal{O}(n)$.
10. Найти минимальный шаблон из букв латинского алфавита и символов “?” и “*” такой, что под него не подходит первая строка, но при этом подходит вторая. $\mathcal{O}(n^3)$.
11. Даны две параллельных прямых (река). В реке есть n островов (точек). Мы хотим провести по реке корабль, представляющий собой открытый круг радиуса R , так, чтобы он не задел ни одного острова. Найти максимальный R , при котором это еще возможно, за $\mathcal{O}(n^2)$.
12. Есть взвешенный неорграф. Найти путь из s в t такой, что сумма двух максимальных ребер на пути минимальна. $\mathcal{O}((V + E) \cdot \text{poly}(\log))$.
13. Есть взвешенный неорграф. Найти путь из s в t такой, что сумма трёх максимальных ребер на пути минимальна. $\mathcal{O}((V + E) \cdot \text{poly}(\log))$.

14. Дан взвешенный граф с положительными весами, в вершинах его стоят числа. Требуется доставить все числа в вершину 0. За то, чтобы провести число a по ребру веса w надо заплатить сумму aw . Если два числа находятся в одной вершине, то их можно слить, заменив на максимум. Постройте алгоритм доставки, дающий константное приближение к оптимальному ответу за полиномиальное время.